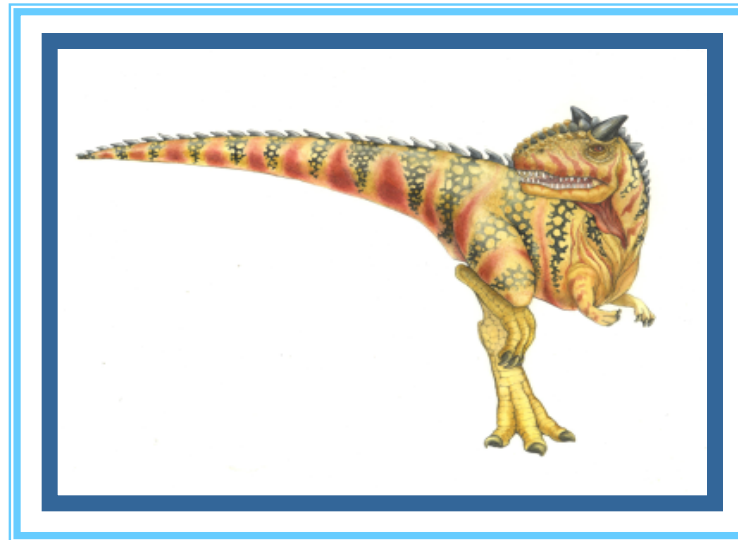


Chapter 9:

File-System Interface





Chapter 9: File-System Interface

- File Concept
- Access Methods
- Disk and Directory Structure
- File-System Mounting
- Protection





Objectives

- To explain the function of file systems
- To describe the interfaces to file systems
- To discuss file-system design tradeoffs, including access methods, file sharing, file locking, and directory structures
- To explore file-system protection





File Concept

- Uniform logical view of information storage (no matter the medium)
- OS abstracts from physical properties into a logical storage unit, the **file**
- Files mapped onto physical devices, usually nonvolatile
- File is a collection of related information
 - Smallest allotment of nameable storage
- Contiguous logical address space
- Types:
 - Data
 - 4 numeric
 - 4 character
 - 4 binary
 - Program





- A **text file** is a sequence of characters organized into lines (and possibly pages).
- A **source file** is a sequence of functions, each of which is further organized as declarations followed by executable statements.
- An **executable file** is a series of code sections that the loader can bring into memory and execute.





File Attributes

- **Name** – only information kept in human-readable form
- **Identifier** – unique tag (number) identifies file within file system
- **Type** – needed for systems that support different types
- **Location** – pointer to file location on device
- **Size** – current file size
- **Protection** – controls who can do reading, writing, executing
- **Time, date, and user identification** – data for protection, security, and usage monitoring
- Information about files are kept in the directory structure, which is maintained on the disk
 - Typically file's name and identifier
 - 4 Identifier locates other file attributes
- Attributes may be > 1KB
- Directory structures may be > 1MB





File Operations

- File is an **abstract data type**
- Operations include the following (and usually more)
- **Create** – find space, add entry to directory
- **Write** – write data at **current file position pointer** location and update pointer
- **Read** – read file contents at pointer location, update pointer
- **Reposition within file (seek)** – change pointer location
- **Delete** – free space and remove entry from directory
- **Truncate** – delete data starting at pointer





Open Files

- $\text{Open}(F_i)$ – allow process to access a file
 - Returns a **file handle** for system call reference to the file
 - Search the directory structure on disk for entry F_i , and move the content or cache some of entry to memory
- $\text{Close}(\text{file handle})$ – end processes' access to the file
 - Move the content of entry F_i in memory to directory structure on disk





File Types – Name, Extension

file type	usual extension	function
executable	exe, com, bin or none	ready-to-run machine- language program
object	obj, o	compiled, machine language, not linked
source code	c, cc, java, pas, asm, a	source code in various languages
batch	bat, sh	commands to the command interpreter
text	txt, doc	textual data, documents
word processor	wp, tex, rtf, doc	various word-processor formats
library	lib, a, so, dll	libraries of routines for programmers
print or view	ps, pdf, jpg	ASCII or binary file in a format for printing or viewing
archive	arc, zip, tar	related files grouped into one file, sometimes com- pressed, for archiving or storage
multimedia	mpeg, mov, rm, mp3, avi	binary file containing audio or A/V information





Access Methods

- **Sequential Access** – tape model of a file

read next

write next

reset

- **Direct Access** – random access, relative access

read n

write n

position to n

read next

write next

rewrite n

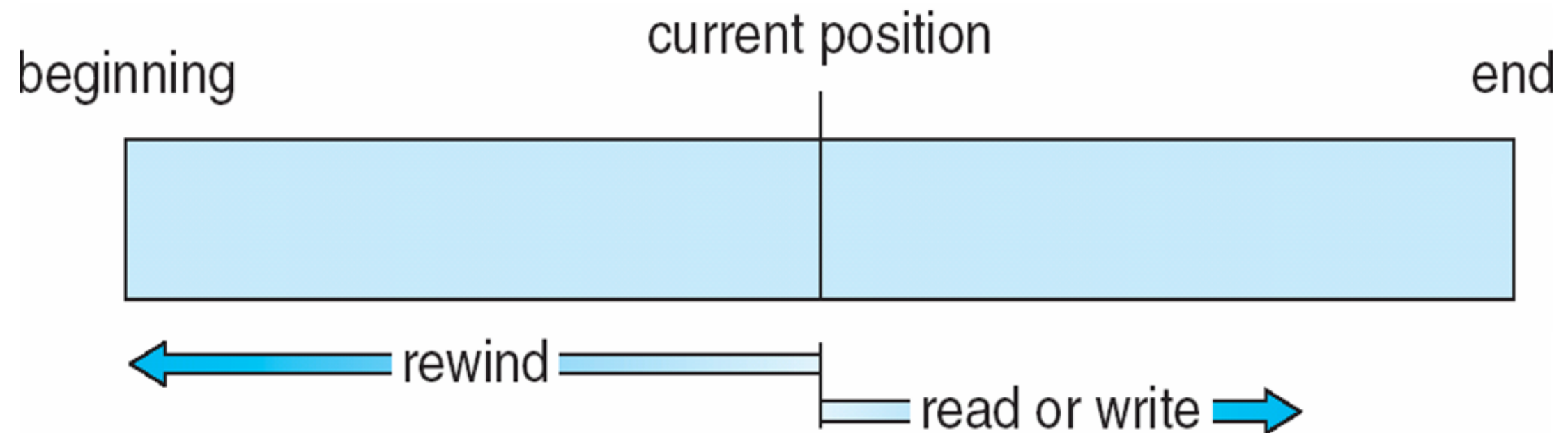
n = relative block number

- Can accommodate structured data in file by mapping record number to block number



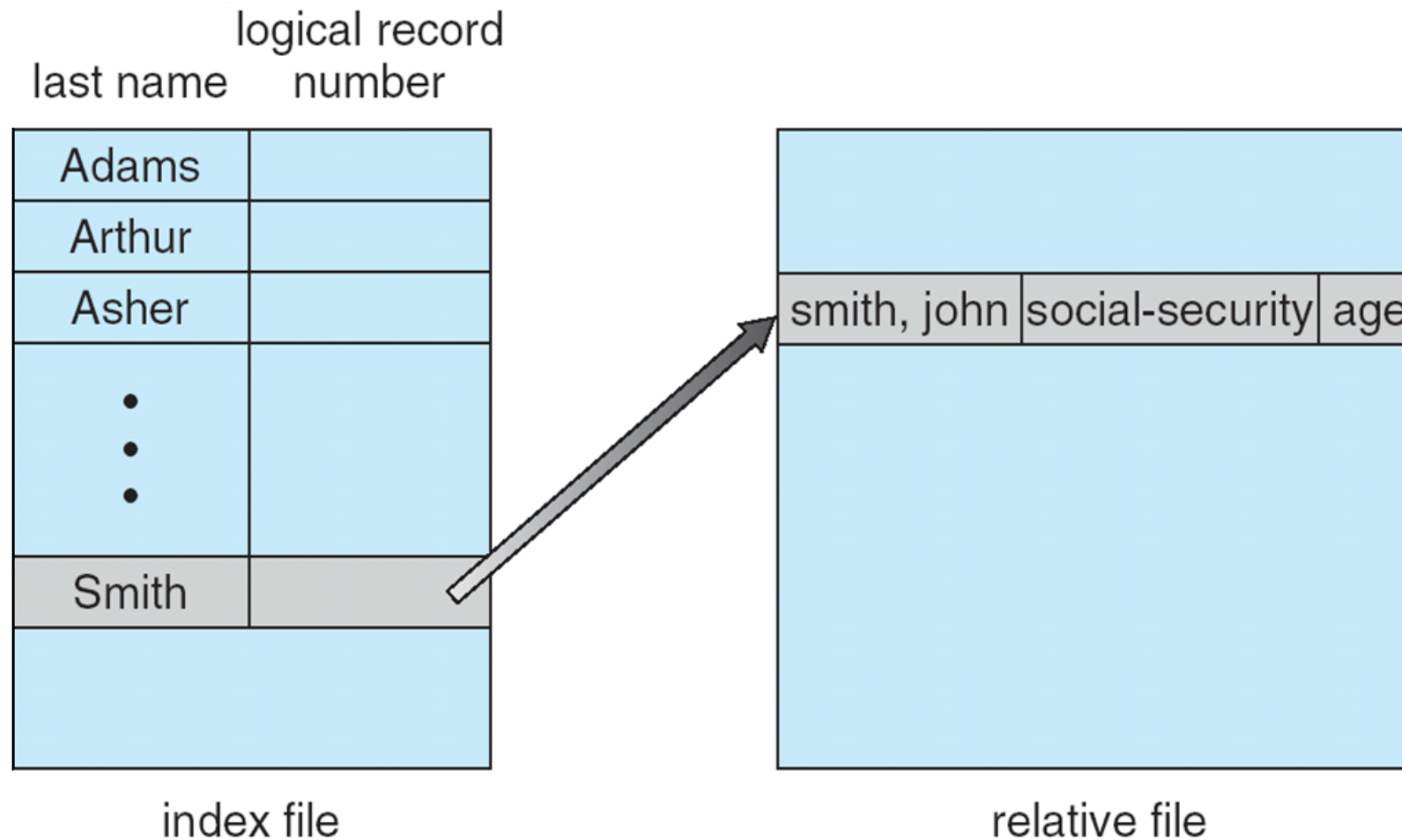


Sequential-access File





Example of Index and Relative Files





Directory Overview

- Directory similar to symbol table translating file names to their directory entries
 - Can be organized in many ways
- Organization needs to support operations including:
 - Search for a file or multiple files
 - Create a file
 - Delete a file
 - List a directory
 - Rename a file





Directory Organization

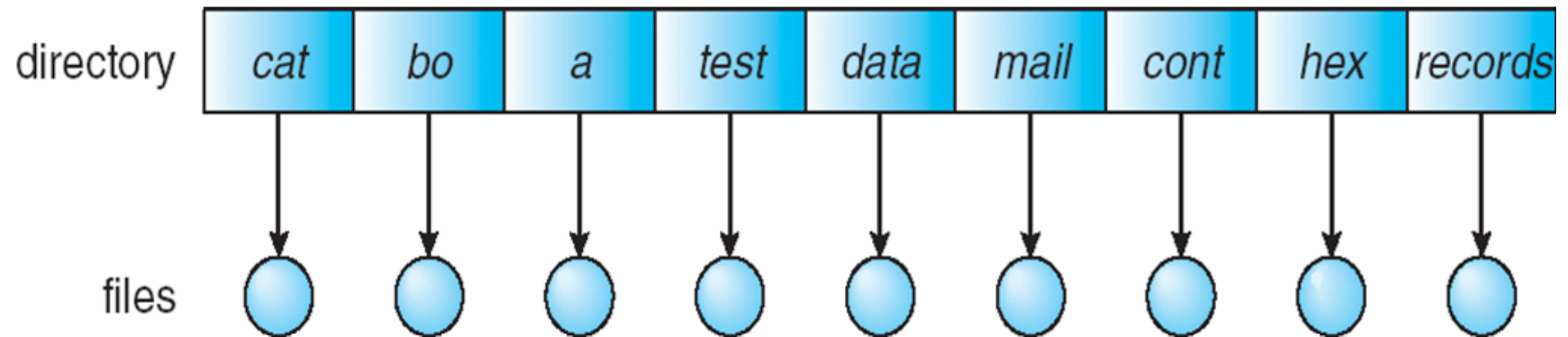
- Should have the features
 - Efficiency – locating a file quickly
 - Naming – convenient to users
 - 4 Two users can have same name for different files
 - 4 The same file can have several different names
 - Grouping – logical grouping of files by properties, (e. g., all Java programs, all games, ...) or arbitrarily





Single-Level Directory

- A single directory for all users



Naming problem

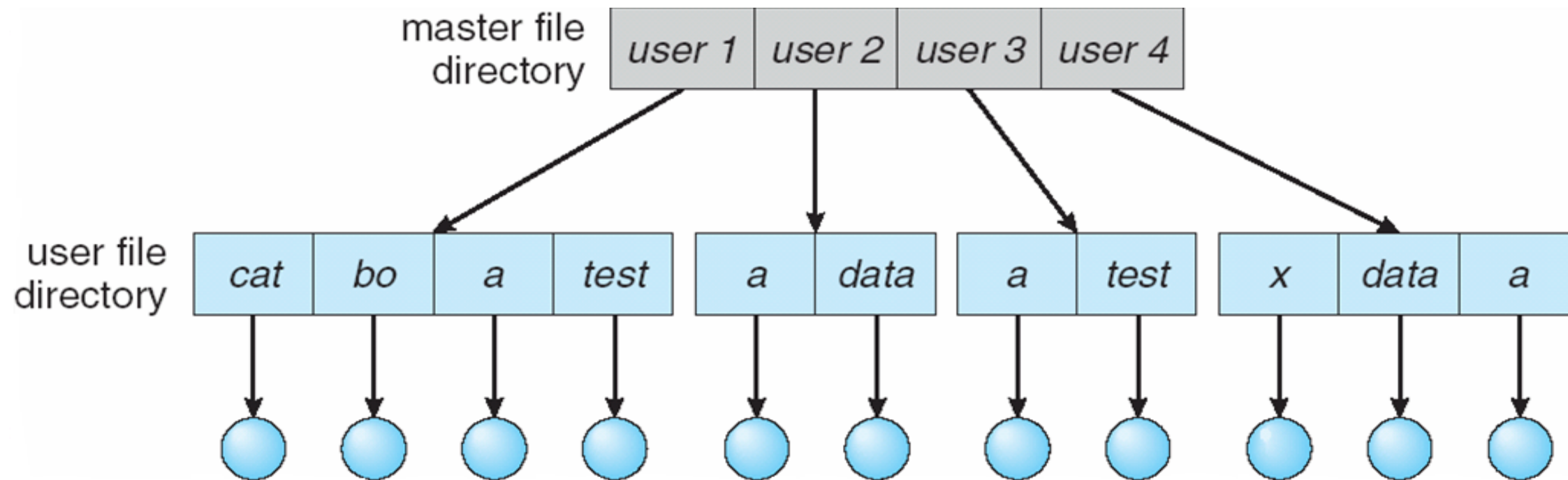
Grouping problem





Two-Level Directory

- Separate directory for each user



- Path name
- Can have the same file name for different users
- Efficient searching





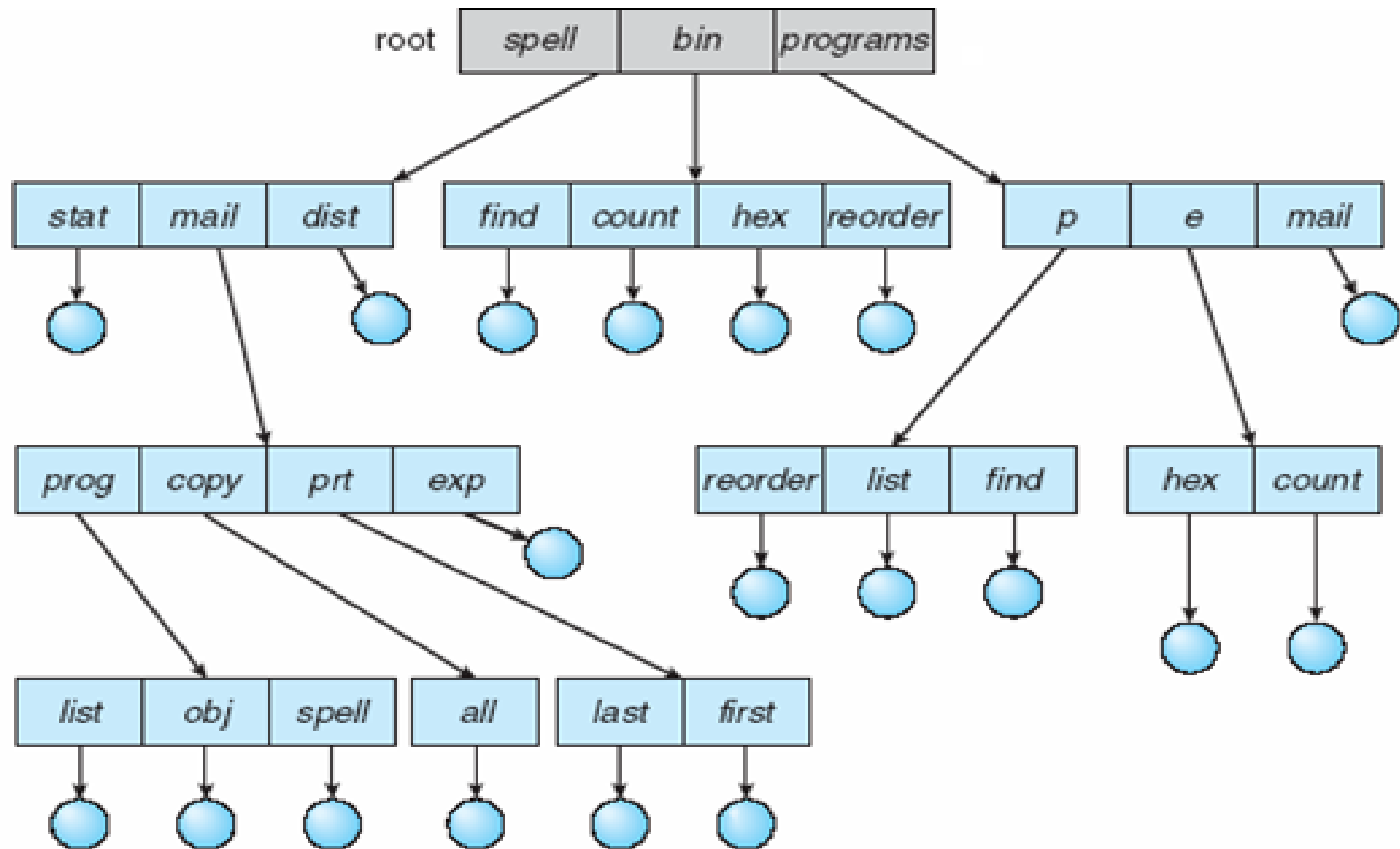
Added Directory Concepts

- Many variations, but some components essential
- Idea of **current directory** – default location for activities
- Now need a **path** specification
 - If file is in current directory, just name it
 - If in another directory, must specify by more detailed name
 - Also need way to specify different filesystems
 - MS-DOS gives letter to each volume, “\” separates directory name from file name – C:\userb\test





Tree-Structured Directories





Protection

- File owner/creator should be able to manage **controlled access**:
 - What can be done
 - By whom
 - But never forget physical security
- Types of access
 - **Read**
 - **Write**
 - **Execute**
 - **Append**
 - **Delete**
 - **List**
 - Others can include renaming, copying, editing, etc

